
WHITE PAPER · VERSION 2.0

DominusOS

The Human-Governed
AI Hypervisor

Author Mark Lord, Dominus Foundry

Date March 2026

Contact mark@dominusfoundry.com

Web dominusos.ai

© 2026 Dominus Foundry. All rights reserved. CC BY-ND 4.0

ABSTRACT

AI systems increasingly execute consequential actions within production infrastructure — managing workflows, processing data, communicating with stakeholders, and allocating resources. Yet governance of these systems remains structurally decoupled from execution: reactive, policy-based, and dependent on configuration rather than architecture. The gap between capability and accountability widens with each new agent framework, each new integration, each new autonomous process.

DominusOS introduces a human-governed AI hypervisor: a control layer beneath AI execution that mediates all system actions through a unified governance architecture. Built on a microkernel with a mandatory syscall gate, capability-based authority model, event-sourced determinism, tenant isolation, and federated multi-kernel orchestration, the system enforces structural guarantees that monitoring and policy alone cannot provide.

The system is in production today — orchestrating multiple virtual employees across federated domain kernels, governed business workflows, a structured knowledge layer unifying decisions and doctrine across domains, native desktop and mobile operator applications, cryptographically sealed constitutional governance, autonomous intelligence that learns and proposes within covenant constraints, and numerous autonomous scheduled operations — all under explicit human authority with split-key governance.

This paper presents the architectural thesis, core components, federation model, intelligence architecture, structural guarantees, production evidence, and roadmap for DominusOS — including planned extension to IoT and autonomous edge environments.

Keywords: AI hypervisor · syscall gate · capability-based authority · deterministic execution · agent orchestration · auditability · observability · tenant isolation · microkernel · structured cognition · knowledge graphs · federated governance · autonomous intelligence · IoT governance

1

The Convergence Problem

The software industry has passed through a series of abstraction phases, each solving one structural problem while creating the conditions for the next.

ERA	ABSTRACTION	SOLVED	LEFT UNSOLVED
Monolith	Centralized control	Integration	Scalability
SaaS	Specialization	Scalability	Data fragmentation
API Economy	Composability	Cross-vendor integration	Dependency complexity
AI Copilots	Local reasoning	Per-task intelligence	Session-scoped memory
Agents	Distributed execution	Task automation at scale	Authority, coherence
Tool Sprawl	Surface coverage	Feature access	Governance

Each phase solved a real problem. None solved the problem that emerges when all of them operate simultaneously: the absence of a governed execution substrate that unifies execution, memory, authority, and observability into a single coherent layer.

This is not a feature gap. It is a structural gap. Modern organizations run dozens of AI-enabled tools across sales, operations, finance, client services, and internal processes. Each tool has its own data store, its own logic, its own failure modes. Intelligence gathered in one domain does not inform decisions in another. Decisions made last quarter cannot be queried, versioned, or connected to outcomes. Doctrine — the standing principles that govern how an organization operates — exists in documents that no system reads, enforces, or versions.

The result is that as AI capability increases, organizational coherence does not increase with it. More automation produces more activity, more data, more surfaces — and less structured understanding. The current stack can execute. It cannot remember, learn, or govern execution in a way that compounds over time.

The absence of a governed execution substrate is the limiting factor in the current AI stack. The next abstraction layer is not another tool. It is a hypervisor.

2

Architectural Thesis

The structural gap in the current AI stack is not a missing tool — it is a missing layer.

AI systems operate with growing autonomy but without a substrate that enforces governance at the point of execution. Current approaches attempt to govern after the fact: monitoring dashboards observe what happened; policy engines advise what should happen; audit logs record what did happen. None intervene architecturally where execution occurs.

DominusOS advances a thesis: **governance must move beneath runtime**. Rather than wrapping AI systems in external policy, DominusOS positions a hypervisor layer between AI processes and the infrastructure they act upon. Every action — every write, every communication, every resource commitment — passes through a syscall gate that enforces scoped, revocable capabilities before execution proceeds.

Authority is not assumed. It is granted, bounded, time-limited, and auditable. No process inherits ambient authority from its environment.

The prevailing direction in the AI tooling ecosystem is toward greater autonomy: more tools, faster execution, longer chains of reasoning, less human involvement. This trajectory optimizes for capability. DominusOS optimizes for accountability.

The difference is not philosophical. It is architectural. A system optimized for capability asks: "How can the agent accomplish the goal with the fewest constraints?" A system optimized for accountability asks: "How can the agent accomplish the goal while preserving the operator's ability to understand, verify, and intervene at every step?" These are different design problems, and they produce different architectures.

This is the distinction between a governed system and a monitored one. Monitoring records what happened. A hypervisor determines what is permitted to happen.

3

Core Architecture

DominusOS is built on five structural components that form the kernel layer.

3.1 Syscall Gate

The mandatory mediation point for all AI execution. No process writes, communicates, deploys, or commits resources without passing through the gate. The gate enforces authority checks against the process's capability set, logs the action to an immutable event ledger, and applies governance rules before permitting execution. The gate is not optional and cannot be bypassed — it is the only path from intent to action.

3.2 Capability-Based Authority

Every process operates within an explicit capability envelope. Capabilities define what a process may do (scope), for how long (time-bound), and under whose authority (attribution). Capabilities are granted by human operators, are transitively revocable, and follow the principle of least authority. A process cannot access any resource or perform any action for which it does not hold an explicit, unexpired capability token.

Having a credential to an API does not constitute permission to use it. Authority is granted, scoped, and recorded as a first-class element of the system's execution model. At any point, for any action the system has taken, it is possible to answer the question: "Who authorized this, and under what terms?"

3.3 Event-Sourced Determinism

Every state change in the system is captured as an immutable, sequenced event. The complete system state can be reconstructed by replaying the event log from any point. This provides replay determinism: any execution sequence can be audited, verified, and reproduced. There are no silent state mutations, no unrecorded side effects, no gaps in the attribution chain.

Observability is not optional logging that can be disabled for performance. It is a structural property of the execution layer. The system does not execute work that it cannot observe. This record is not for debugging. It is for accountability.

3.4 Tenant Isolation

Organizational domains execute within isolated boundaries. No process in one domain can access another domain's data, capabilities, or execution context without explicit cross-domain authority grants. Isolation limits blast radius, prevents cascade failures, and ensures that a fault or compromise in one domain cannot propagate to others.

3.5 Microkernel Architecture

The kernel is minimal by design. It manages process scheduling, capability enforcement, the syscall gate, and the event ledger. All domain logic, integrations, drivers, and services run in user space outside the kernel boundary. A fault in any driver or integration cannot crash the kernel or affect other subsystems. The design minimizes the trusted computing base and maximizes fault containment.

Federation Architecture

DominusOS extends beyond a single kernel into a federated multi-kernel architecture. Each organizational domain operates its own kernel — with its own isolated data store, its own audit chain, its own governance rules — while remaining governed by a sovereign layer that holds constitutional authority.

4.1 Sovereign Kernel

The sovereign kernel sits above all domain kernels. It holds the constitutional governance canon, issues trust passports, manages the federation registry, and aggregates abstracted intelligence from all domains. No raw domain data crosses the federation boundary — only abstracted signals. The sovereign kernel cannot be overridden by any domain kernel. Constitutional rules flow downward. Intelligence flows upward.

4.2 Domain Kernels

Each business domain operates its own kernel instance with full tenant isolation. Domain kernels inherit constitutional governance articles from the sovereign at boot. They maintain their own audit chains, their own capability registries, and their own operational data. Domain kernels register with the sovereign federation registry on startup and emit regular heartbeat signals for health monitoring.

4.3 Federation Event Bus

Cross-kernel communication occurs through a structured event bus with explicit routing rules. Events carry provenance, capability attestation, and trust passport validation. No domain kernel can emit events that exceed its granted federation authority. The event bus enables intelligence aggregation without compromising domain isolation.

4.4 Trust Passports

The sovereign kernel issues trust passports to domain kernels and entities operating within them. Trust passports encode what a kernel or entity is authorized to do within the federation. Passports are time-bounded, scopable, and revocable. Trust is not inherited — it is issued, verified, and maintained through continuous signal evaluation.

4.5 Master Cortex

The federated knowledge graph sits at the sovereign layer and aggregates abstracted signals from all domain kernels. It does not hold raw domain data. It holds patterns, correlations, and intelligence derived from cross-domain signal analysis. A behavioral shift detected in one domain can surface as a relevant signal in another domain's operational context. The intelligence is combinatorial: each new domain added to the federation multiplies the intelligence available to every domain already connected.

Intelligence Architecture

DominusOS includes a governed intelligence layer that learns, proposes, and — within explicit bounds — acts autonomously. Intelligence is not bolted on after governance. It operates within governance as a first-class governed process.

5.1 Event Processing Substrate

A continuous event processing layer performs sliding-window aggregation, sequence detection, frequency analysis, and anomaly scoring across all kernel events. This substrate produces behavioral baselines — statistical models of normal system and entity behavior — that enable drift detection without requiring predefined rules for what constitutes abnormal.

5.2 Learning Manifest

Before the intelligence layer is activated, a sealed manifest defines exactly what patterns the system is authorized to detect. The system can only find what it is instrumented to find. Pattern detection targets are documented, version-controlled, and subject to the same governance authority as any other system capability. The manifest includes entity behavioral drift, trust signal velocity, knowledge acquisition anomalies, cross-domain correlations, and federation event bus patterns.

5.3 Autonomic Planner

The autonomic planner follows the MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) pattern. It monitors behavioral baselines, analyzes deviations, plans proposed responses, and — at authorized tiers — executes responses. All proposals carry stated evidence and are subject to a tiered authority model:

Tier 1: Auto-propose, operator reviews. Trust weight adjustments, priority reordering, knowledge node changes.

Tier 2: Propose with evidence required. Governance parameter changes, entity authority adjustments.

Tier 3: Propose with governance gate. Changes affecting federation topology, cross-kernel trust, or constitutional governance.

No tier permits unilateral action that affects constitutional governance. The operator's authority always outpaces the system's ability to act.

5.4 Evolution Engine

The system improves over time through a governed evolution process. Proposed optimizations — workflow improvements, behavioral governance refinements, scheduling adjustments — are generated by the intelligence layer, reviewed under explicit authority, and executed with full rollback capability. Every evolution is versioned, every outcome is measured against the proposal's stated hypothesis, and every change can be reversed.

Evolution inside governance, not outside it.

5.5 Counsel Layer

When the system has accumulated sufficient operational history, it produces structured counsel — evidence-cited briefs delivered from the system to the operator. Counsel surfaces the operator's own patterns: behavioral trends, cross-domain correlations, anomalies accumulating below threshold, and outcomes of previously executed evolution proposals. Every claim in a counsel brief cites specific evidence — event counts, baseline deviations, confidence scores, time windows. Counsel is not a dashboard. It is a letter from the system to its steward, written in plain language, backed by evidence.

5.6 Deep Temporal Memory

The intelligence layer maintains queryable access to the full depth of operational history. Natural language queries can interrogate decisions, patterns, and behavioral trajectories across the complete timeline. When a current operational context matches a prior pattern, the system surfaces the relevant history without being asked. Intelligence compounds over time because nothing is discarded and everything is connected.

6

Constitutional Governance

DominusOS implements a cryptographically enforced constitutional layer that cannot be modified by any automated process.

6.1 Foundation Stone

The kernel's invariant constants are embedded in the boot sequence. On every kernel start, the foundation is verified before any other process loads. If the foundation has been tampered with, the kernel refuses to start. These constants are not configurable — they are architectural invariants.

6.2 Constitutional Canon

The constitutional governance rules — the articles that define what the system may and may not do at the deepest level — are stored in a sealed canon directory. The canon defines authority boundaries, governance gates, audit requirements, and the non-negotiable operational principles that no process, including privileged ones, may violate.

6.3 Cryptographic Canon Seal

The canon and foundation are cryptographically hashed into a composite seal. On every kernel boot, the seal is recomputed and verified. If any canon file has been modified, the hash changes and the kernel halts. The seal cannot be regenerated without the covenant key.

6.4 Split-Key Authority

The covenant key that authorizes canon modifications is split using threshold cryptography. No single individual holds the complete key. Modification of the constitutional governance layer requires the physical presence and consent of multiple designated human stewards. The key is reconstructed only for the modification ceremony, never persists in memory or on disk, and is re-split immediately after use. This ensures that the deepest governance layer of the system requires human consensus to change — not just human approval, but human collaboration.

6.5 Multi-Operator Sovereignty

The system supports multiple human operators with differentiated authority. Operators can hold domain-specific sovereignty — full authority over their own domain kernel's governance within the bounds of the constitutional canon. Split-key governance ensures that constitutional changes require consensus among designated stewards. Domain operators can make domain-level governance changes independently without requiring sovereign-level approval, while sovereign-level changes require the full split-key ceremony.

Structural Guarantees

DominusOS provides eight structural guarantees enforced by architecture, not by policy:

1. **Replay Determinism.** Any execution sequence can be replayed from the event log and will produce identical, verifiable results. Full audit reconstruction from any point in system history.
2. **Capability Enforcement Integrity.** No process can exceed its granted capabilities. Enforcement is mandatory and architectural. Scope violations are structurally difficult by design, not merely detectable.
3. **Tenant Isolation Integrity.** No cross-tenant data access, capability escalation, or execution context leakage without explicit authority grants. Domain boundaries are enforced at the kernel level.
4. **Revocation Cascades.** When a capability is revoked, all capabilities derived from it are transitively and immediately revoked. Authority cannot persist after withdrawal. The operator's ability to halt always outpaces the system's ability to act.
5. **Driver Fault Containment.** A failure in any driver, integration, or user-space service cannot propagate to the kernel or affect unrelated subsystems. The system fails safe by design.
6. **Canon Immutability.** The system's constitutional governance rules cannot be modified by any process, including privileged ones. The governance framework is cryptographically sealed and can only be amended through a formal ceremony requiring split-key human authority.
7. **Federation Isolation.** No domain kernel can access another domain kernel's raw data, escalate its federation authority, or emit events exceeding its trust passport scope. Cross-domain intelligence is aggregated through abstracted signals only.
8. **Governed Intelligence.** The intelligence layer cannot act outside its sealed learning manifest. Autonomic proposals are tiered, evidence-cited, and subject to operator authority. The system cannot modify its own constitutional governance under any circumstance.

The Structural Argument

Constraint is what makes intelligent systems stable.

Autonomy without constraint produces entropy. Systems that act without boundaries eventually act in ways that conflict with organizational intent.

Learning without versioning produces corruption. Systems that overwrite previous state cannot distinguish between current belief and prior understanding.

Execution without structured memory produces repetition. Intelligence is generated but never accumulated. Each interaction starts from zero.

Tool proliferation without substrate produces incoherence. Each new tool creates a new data silo, a new failure surface, and a new governance gap.

Monitoring without mediation produces the illusion of control. Observability tells you what happened. Only a governance layer can determine what is permitted to happen.

Intelligence without governance is corruption waiting to happen. A system that can learn and act but cannot be constrained, audited, or halted is a liability, not an asset.

The stability of intelligent systems depends on structural constraint, governed memory, non-destructive evolution, and constitutional immutability. These properties must be enforced beneath runtime, not applied above it.

Current State

DominusOS is not a proposal or a prototype. The system is deployed and running in production, orchestrating governed AI operations for a multi-domain service business:

- **Multiple Virtual Employees** executing governed workflows daily across email, document processing, compliance enforcement, and operational intelligence.
- **Federated Domain Kernels** operating with full tenant isolation, independent audit chains, and sovereign-inherited constitutional governance.
- **Structured Knowledge Layer** unifying decisions, projects, doctrine, contacts, and operational insights as versioned, graph-connected nodes with full provenance.
- **Governed Business Operations** with full-funnel attribution, composite engagement scoring, automated follow-up enforcement, and stale data detection.
- **Operator Assembly Surface** for structured cognition — decision modeling, project lifecycle management, and version-controlled doctrine.
- **Native Operator Applications** on desktop and mobile with live process visibility, tab-based task management, and direct kill authority.
- **Autonomous Scheduled Operations** running under governance with full attribution chains and immutable activity ledger.
- **Governed Intelligence Layer** detecting patterns, proposing actions with evidence, and evolving within explicit authority bounds.
- **Cryptographically Sealed Canon** with split-key authority protecting constitutional governance from unauthorized modification.
- **System-Level Kill Switch** that halts all AI execution instantly and visibly. One action. Total halt.

The hypervisor architecture is a deployable pattern. Any multi-domain operation that requires accountability, structured cognition, cross-domain intelligence, and governed AI execution can run a version of this system.

Roadmap

Current: Enterprise AI Governance

Founding deployments with organizations that require governed AI operations, structural accountability, and auditable execution across multiple domains. The system is live and proving the pattern in production.

Near-Term: Multi-Tenant Platform

Scaling the hypervisor to support multiple organizations, each operating within isolated tenant boundaries with independent governance configurations, authority structures, knowledge layers, and constitutional canons.

Long-Term: Autonomous Infrastructure and IoT

The capability-based authority model, syscall gate, and tenant isolation architecture are designed to extend to IoT and autonomous edge environments. When autonomous systems execute actions in the physical world — controlling devices, managing facilities, operating equipment — the need for bounded authority, deterministic execution, and immediate halt capability becomes critical. The IoT extension layer is designed into the current architecture, with the syscall gate and capability model accommodating device-level authority grants, edge-local governance enforcement, and physical-world action mediation.

AI governance today. Autonomous infrastructure tomorrow.

Competitive Landscape

Most organizations assemble a patchwork of tools that each solve one problem. None of them solve the governance problem.

Automation platforms connect systems — they move data between services. They do not enforce authority boundaries, maintain audit-grade provenance, or provide a single kill switch when something goes wrong. When AI agents run inside them, no one controls the scope.

Agent frameworks let AI call functions and produce outputs. Most lack hard authority boundaries, durable process identity, and the ability to trace any outcome to the human who approved it. When the agent drifts, no structural failsafe exists.

Traditional CRMs track activity and pipeline. They do not provide immutable ledgers, full-funnel attribution from origin action to closed deal, composite scoring with governed signals, or learning loops that improve under governance. History can be rewritten. The chain cannot be audited.

Knowledge and note tools store documents and databases. They do not enforce decision modeling, version-control doctrine, connect knowledge to execution outcomes, or distinguish between human-verified facts and AI-generated suggestions. Information goes in. Intelligence does not come out.

Observability dashboards show you what happened. They do not control what is allowed to happen next. They observe — they do not govern.

DominusOS operates at the hypervisor layer — beneath execution, above infrastructure. A mandatory syscall gate, capability-based authority, event-sourced determinism, federated multi-kernel governance, governed intelligence, and structural kill authority. Execute, govern, prove, halt — in one system. Not bolted on. Architected in.

References

Citation: Lord, M. (2026). DominusOS: The Human-Governed AI Hypervisor. Dominus Foundry. <https://dominusos.ai>

Copyright © 2026 Dominus Foundry. All rights reserved.

Licensed under CC BY-ND 4.0. You may share and cite this work with attribution. Modifications are not permitted.

Website: dominusos.ai

Specification Repository: github.com/markl2305/dominus-os

Company: Dominus Foundry

Author: Mark Lord (mark@dominusfoundry.com)